B 1.1.2 Computational thinking

B 1.1.2 The fundamental concepts of computational thinking.

Computational thinking is a structured approach to problem-solving. It draws on four interrelated concepts, decomposition, pattern recognition, abstraction, and algorithmic design, which are applied together, and often iteratively, to analyse problems and design solutions. Though rooted in computer science, these concepts underpin systematic thinking across every discipline.

Last updated: 25 May 2026

Key Questions

B 1.1.2 — Describe the fundamental concepts of computational thinking.

- What are the four fundamental concepts of computational thinking and how do they relate to one another?
- How is computational thinking applied when approaching a real problem, and why is the process iterative rather than linear?
- In what ways do abstraction and decomposition differ, and when would you apply each?

IB Command terms

Describe
Give a detailed account of the features or characteristics of each concept. Name, define, and give an example.

What to include
For each concept: a clear definition, a concrete CS-relevant example, and how it connects to the others.

What is computational thinking?

Every piece of software you use, from a search engine to a navigation app, was built by someone who had to take a complex, complex, unstructured problemand make it solvable by a machine. Computational thinking is the set of skills that makes that possible.

It is not the same as programming. You can apply computational thinking to any problem, scheduling a tournament, designing a database, or planning a scientific experiment, whether or not you ever write a line of code. What it gives you is a disciplined way of thinking: breaking problems apart, spotting structure, removing irrelevant detail, and designing precise solutions.

The four concepts below are not a checklist to work through once. In practice, they are applied repeatedly and in combination, refining a solution as understanding of the problem deepens.

The four concepts of computational thinking

Decomposition

Breaking a problem into manageable sub-problems

> Decomposition means dividing a complex problem into smaller, independent sub-problems that can each be understood, solved, and tested separately. The sub-problems may be further decomposed until each part is simple enough to address directly.

Examples

- **Search engine:** decomposed into crawling the web, indexing pages, ranking results, and serving responses — each a distinct engineering problem.
- **Ride-hailing app:** location tracking, driver matching, route calculation, fare computation, and payment processing are all separate subsystems.
- **Sorting algorithm:** a merge sort decomposes a list into halves, sorts each, then merges — the same logic applied recursively.

Connects to: Once decomposed, sub-problems can be tackled algorithmically, and patterns across sub-problems can be exploited to avoid redundant work.

Pattern recognition

Identifying structure, similarity, and regularity

> Pattern recognition involves identifying similarities, regularities, or recurring structures within a single problem or across different problems. Recognising a pattern means a solution or approach can be generalised rather than rebuilt from scratch.

Examples

- **Spam detection:** patterns in email headers, sender reputation, and keyword frequency recur across spam messages. A model trained on these patterns can generalise to new emails.
- **Debugging**: a null pointer exception appearing in multiple modules follows a pattern that points to a common root cause, missing initialisation.
- **Algorithm design:** recognising that two problems share the same underlying structure (e.g. shortest path and minimum spanning tree) allows the same algorithmic technique to be reused.

Connects to: Patterns identified here directly inform abstraction. If the same pattern appears in multiple contexts, it signals that a generalised abstraction is possible.

Abstraction

Focusing on what matters, hiding what doesn't

> Abstraction means representing a problem or system by retaining only the details that are relevant to the task and deliberately omitting those that are not. A good abstraction makes the problem manageable without losing the features that matter.

Examples

- **Data modelling:** a student record abstracts a real person to the fields that matter for the system — ID, name, grades, attendance — ignoring hair colour, height, and shoe size.
- **Functions and APIs:** a sort() call abstracts away the implementation — you specify what to sort, not how comparisons and swaps work.
- **Network model:** the OSI model abstracts a physical network into seven layers; software engineers work at the application layer without needing to understand signal voltages.

Connects to: Abstraction shapes what goes into an algorithm — it determines which inputs matter and what the algorithm needs to produce, keeping the design focused.

Algorithmic design

Designing precise, repeatable solutions

> An algorithm is a finite, ordered sequence of unambiguous instructions that solves a problem or achieves a goal. Algorithmic design is the process of constructing that sequence — specifying not just what needs to happen, but exactly how and in what order.

Examples

- **Binary search:** a precise sequence comparison to target the midpoint, eliminate half the list, repeat — that works correctly on any sorted list of any size.
- **Login validation:** retrieve stored hash → hash input → compare → grant or deny access → log attempt. Each step is unambiguous and must occur in order.
- **Recommendation system:** an explicit sequence of steps that collect interaction data, build a user profile, score candidate items, filter, rank, and return top N constitutes the algorithm, regardless of how complex each step becomes.

Connects to: Algorithmic design operates on the abstracted problem and typically applies to each sub-problem identified by decomposition. The algorithm may also encode a recognised pattern as a reusable procedure.

Computational thinking in action

The four concepts are rarely applied in isolation or in strict sequence. Consider the problem of designing a plagiarism-detection system for a school as a realistic, bounded problem that makes the thinking visible.

Problem: Given a student submission and a reference corpus of past work and web content, determine whether the submission contains unoriginal text and report the extent of similarity.

Decomposition	Break the system into sub-problems: (1) pre-process and normalise text, (2) generate a fingerprint or signature for each document, (3) compare fingerprints against the corpus, (4) identify and highlight matching regions, (5) produce a similarity score and report. Each is now an independent engineering task.
Pattern recognition	Plagiarised passages tend to share n-gram sequences (short overlapping word sequences). This recurring pattern of identical or near-identical n-grams appearing in two documents is the signal the system is designed to detect. Recognising this guides the choice of fingerprinting technique.
Abstraction	Represent each document not as raw text but as a set of hashed n-gram values. This abstracts away font, spacing, synonyms, and superficial edits, retaining only the structural similarity that indicates copying. The student's name, submission date, and word count are irrelevant to detection and are excluded from the model.

Algorithmic design	For each sub-problem, design a precise algorithm: tokenise → remove stop words → generate n-grams → hash each → store in a set. Then compare: compute the Jaccard similarity (a standard measure of how much two sets overlap) of the two sets (intersection size ÷ union size). If similarity exceeds a threshold, flag and return matching regions. This is deterministic and repeatable for any input.

Why this is iterative: After testing, you may find that the threshold produces too many false positives (common phrases flagged as plagiarism). This sends you back to abstraction, perhaps stop-word lists need extending, and then back to algorithmic design to adjust the threshold logic. Computational thinking cycles do not terminate at step four.

Summary

Decomposition Divide a complex problem into smaller, independently solvable sub-problems. *Search engine → crawl / index / rank / serve*	**Pattern recognition** Identify recurring structures or similarities to generalise a solution. *Spam detection via recurring keyword and header patterns*
Abstraction Retain only the details relevant to the problem; discard the rest. *A student record omits height — includes ID, grades, attendance*	**Algorithmic design** Construct a finite, unambiguous sequence of steps that solves the problem. *Binary search: compare → eliminate half → repeat*

B 1.1.2 Quizlet

Test your knowledge with these flashcards

Match B 1.1.2 Describe the fundamental concept...

Ready to play?

Match all the terms with their definitions as fast as you can. Avoid wrong matches, they add extra time!

Start game

View this flashcard set Choose a Study Mode

B 1.1.2 Quiz

1

Which is NOT a core concept of computational thinking?

A. Pattern recognition

B. Decomposition

C. Abstraction

D. Randomisation

2

Decomposition is best described as:

A. Finding similarities in data.

B. Creating a set of instructions.

C. Ignoring irrelevant details.

D. Breaking down a problem into smaller parts.

3

Which is an example of pattern recognition?

A. Predicting the weather based on past data.

B. Hiding complex code behind a user interface.

C. Using a map to navigate.

D. Writing a recipe for a cake.

4

Abstraction helps us:

A. Identify errors in code.

B. Solve problems more quickly.

C. Focus on the most important information.

D. Create more complex algorithms.

5

An algorithm is:

A. A random set of instructions.

B. A type of programming language.

C. A specific sequence of steps to solve a problem.

D. A way to visualise data.

6

Which of the following best describes an algorithm?

A. A general idea for solving a problem

B. A finite, ordered sequence of unambiguous steps that produces a result

C. A programming language used to write software

D. A diagram showing how data flows through a system

7

A developer writes a `calculateTax(income)` function that handles all the tax-band logic internally. When another programmer calls the function, they only need to know what it returns, not how it works. Which concept does this illustrate?

A. Decomposition

B. Pattern recognition

C. Algorithmic design

D. Abstraction

8

Which situation benefits from decomposition?

A. Taking a nap.

B. Organising a school fundraising event.

C. Choosing what to eat for breakfast.

D. Watching a movie.

9

A social media algorithm uses pattern recognition to:

A. Show you posts from your friends.

B. Recommend content you might like.

C. Allow you to upload photos.

D. Send you notifications.

10

A programmer notices that three different modules in their code all crash with a null pointer exception under similar conditions. Identifying this as a single underlying fault rather than three separate bugs is an example of:

A. Abstraction

B. Decomposition

C. Pattern recognition

D. Algorithmic design

Report a problem